

The Knuth–Morris–Pratt (KMP) Algorithm

1 Introduction

Main Idea

KMP searches for a pattern $P[1 : m]$ in a text $T[1 : n]$ without rechecking characters unnecessarily. It uses the pattern's own structure to decide where to continue after a mismatch.

Running Time

Preprocessing $\Theta(m)$,	matching $\Theta(n)$,	total $\Theta(n + m)$.
-----------------------------	------------------------	-------------------------

The key precomputed array is the **prefix function** $\pi[1 : m]$. It tells KMP how much of the current match can still be reused.

2 The Prefix Function

Prefix Function

For each $q = 1, 2, \dots, m$,

$$\pi[q] = \max\{k \mid k < q \text{ and } P_k \text{ is a suffix of } P_q\}.$$

Interpretation

If a mismatch occurs after matching q characters, then KMP falls back to

$$q \leftarrow \pi[q].$$

So $\pi[q]$ is the reusable matched length.

3 Worked Example of the Prefix Function

Let

$$P = \text{ababaca}.$$

We compute $\pi[q]$ for each position.

q	$P[q]$	Prefix $P[1 : q]$	$\pi[q]$
1	a	a	0
2	b	ab	0
3	a	aba	1
4	b	abab	2
5	a	ababa	3
6	c	ababac	0
7	a	ababaca	1

Thus:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

4 How KMP Uses the Prefix Function

If q characters of P have matched and the next character fails, KMP does not restart. It falls back to

$$q \leftarrow \pi[q],$$

which keeps the longest prefix that can still be reused.

5 The KMP Matcher Algorithm

KMP Matcher

```

KMP-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  pi = COMPUTE-PREFIX-FUNCTION(P)
4  q = 0
5  for i = 1 to n
6      while q > 0 and P[q+1] != T[i]
7          q = pi[q]
8      if P[q+1] == T[i]
9          q = q + 1
10     if q == m
11         print "Pattern occurs with shift", i - m
12         q = pi[q]
```

Reading the Algorithm

i scans the text and q stores how many pattern characters are currently matched. On a mismatch, lines 6–7 shrink q using $\pi[q]$. On a full match, line 12 again uses $\pi[q]$, so overlapping matches are not missed.

6 Worked Matching Example

Let

$$P = \text{ababaca}, \quad T = \text{abababacaba}.$$

We already know:

$$\pi = [0, 0, 1, 2, 3, 0, 1].$$

Now track the matching process.

Step-by-step

i	$T[i]$	q after processing	Comment
1	a	1	matched $P[1]$
2	b	2	matched $P[2]$
3	a	3	matched $P[3]$
4	b	4	matched $P[4]$
5	a	5	matched $P[5]$
6	b	4	mismatch with $P[6] = c$, fallback to $\pi[5] = 3$, then match b
7	a	5	matched again
8	c	6	matched $P[6]$
9	a	7	full pattern matched
10	b	2	fallback after full match
11	a	3	continue scanning

At $i = 9$, we have:

$$q = 7 = m.$$

So the pattern occurs with shift:

$$9 - 7 = 2.$$

Result

The pattern **ababaca** occurs in the text **abababacaba** at shift

2.

7 Computing the Prefix Function

The prefix function itself is computed efficiently by another linear-time procedure.

Compute Prefix Function

```
COMPUTE-PREFIX-FUNCTION(P)
1  m = P.length
2  let pi[1:m] be a new array
3  pi[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k+1] != P[q]
7          k = pi[k]
8      if P[k+1] == P[q]
9          k = k + 1
10     pi[q] = k
11 return pi
```

8 Meaning of the Prefix-Function Algorithm

Here:

- q is the current position whose prefix value we want,
- k stores the current best candidate for $\pi[q - 1]$.

The algorithm extends the previous best border if possible. If not, it repeatedly falls back using the already computed prefix values.

Key Insight

Just as KMP avoids recomparing text characters, the prefix-function algorithm avoids recomparing pattern characters unnecessarily.

9 Worked Example: Computing π

Let

$$P = \text{ababaca}.$$

We compute π step by step.

Initialization

$$\pi[1] = 0, \quad k = 0.$$

For $q = 2$

Compare:

$$P[k + 1] = P[1] = a, \quad P[2] = b.$$

Mismatch, so:

$$\pi[2] = 0.$$

For $q = 3$

Compare:

$$P[1] = a, \quad P[3] = a.$$

Match, so:

$$k = 1, \quad \pi[3] = 1.$$

For $q = 4$

Compare:

$$P[2] = b, \quad P[4] = b.$$

Match, so:

$$k = 2, \quad \pi[4] = 2.$$

For $q = 5$

Compare:

$$P[3] = a, \quad P[5] = a.$$

Match, so:

$$k = 3, \quad \pi[5] = 3.$$

For $q = 6$

Compare:

$$P[4] = b, \quad P[6] = c.$$

Mismatch, so fallback:

$$k = \pi[3] = 1.$$

Now compare:

$$P[2] = b, \quad P[6] = c.$$

Mismatch again, fallback:

$$k = \pi[1] = 0.$$

Now compare:

$$P[1] = a, \quad P[6] = c.$$

Still mismatch, so:

$$\pi[6] = 0.$$

For $q = 7$

Compare:

$$P[1] = a, \quad P[7] = a.$$

Match, so:

$$k = 1, \quad \pi[7] = 1.$$

Thus:

$$\boxed{\pi = [0, 0, 1, 2, 3, 0, 1].}$$

10 Running-Time Analysis of Compute-Prefix-Function

At first glance, the while loop seems expensive. But overall, the algorithm is linear.

Why?

- k increases only when a match occurs.
- k decreases only through:

$$k = \pi[k].$$

- k can never become negative.

So across the entire algorithm:

- total increases of k are at most $m - 1$,
- total decreases of k are also at most $m - 1$.

Therefore the total running time is:

$$\Theta(m).$$

Conclusion

The prefix function is computed in linear time.

11 Running-Time Analysis of KMP Matcher

The matcher also runs in linear time.

Why?

As the text is scanned:

- i increases from 1 to n ,
- q increases when characters match,
- q decreases only through prefix-function fallback.

Just as before, total increases and decreases of q are both bounded by n .

So total work is:

$$\Theta(n).$$

Thus the overall KMP complexity is:

$$\Theta(n + m).$$

Why is the matching time linear?

The variable q measures how many pattern characters are currently matched. During the whole scan:

- q can increase at most n times, once per text character,
- every fallback sets $q = \pi[q]$, so q strictly decreases,
- q never goes below 0.

Therefore the total number of fallbacks is also at most n . So the matching phase takes:

$$\boxed{\Theta(n)}.$$

Including prefix computation, the total running time is:

$$\boxed{\Theta(n + m)}.$$

12 Prefix-Function Iteration Lemma

A key theoretical result is that repeated application of π lists all proper prefix-suffix lengths.

Lemma 1 (Prefix-Function Iteration Lemma). *For a pattern P , the values*

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

list all lengths $k < q$ such that P_k is a suffix of P_q .

Meaning

If you start at q and repeatedly follow

$$q \rightarrow \pi[q] \rightarrow \pi[\pi[q]] \rightarrow \dots,$$

you visit all valid smaller borders of P_q .

This explains why fallback using π is enough: no possible useful prefix is missed.

Border Chain

The prefix function creates a chain of fallback states:

$$q, \pi[q], \pi[\pi[q]], \dots, 0.$$

KMP follows this chain until it finds a state from which matching can continue.

13 Proof of the Prefix-Function Iteration Lemma

Proof. The value $\pi[q]$ is the largest proper border of P_q . Then $\pi[\pi[q]]$ is the largest proper border of that border, and so on. Therefore

$$\pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots$$

are valid borders of P_q .

If j is any other proper border of P_q , then $j \leq \pi[q]$. If $j < \pi[q]$, then j is a border of $P_{\pi[q]}$, so the same argument continues on $\pi[q]$. Since every step strictly decreases the length, the chain lists all proper borders from largest to smallest. $\square$

Result

The fallback chain lists every reusable border:

$$q, \pi[q], \pi[\pi[q]], \pi[\pi[\pi[q]]], \dots, 0.$$

14 Correctness Idea of KMP

KMP can be viewed as simulating the finite-automaton matcher, but without explicitly storing the full transition table.

Instead of precomputing $\delta(q, a)$ for every state and alphabet character, KMP computes the needed transition on the fly using π .

Core Correctness Idea

After reading $T[1 : i]$, the variable q equals the length of the longest prefix of P that is also a suffix of $T[1 : i]$.

This is the same invariant used by the finite-automaton approach.

So when:

$$q = m,$$

the whole pattern has just matched.

15 Comparison with Previous Methods

Method	Preprocessing	Matching
Naive Matcher	0	$\mathcal{O}((n - m + 1)m)$
Rabin–Karp	$\Theta(m)$	average good, worst-case $\mathcal{O}((n - m + 1)m)$
Finite Automaton	$\mathcal{O}(m \Sigma)$ or more	$\Theta(n)$
KMP	$\Theta(m)$	$\Theta(n)$

Why KMP is Special

KMP keeps the linear-time matching performance of finite automata, but reduces preprocessing from $\mathcal{O}(m|\Sigma|)$ to:

$$\Theta(m).$$